



少年圖靈計畫
Young Turing Program

0_送分題 – Hello World

(30分)

前言

比賽開始了！

趕快驗證一下，

網路是否設定正確？

上傳競賽程式是否順利？

程式解答是否用 STDOUT 輸出？

都沒問題，30分就到手了！繼續 ... 衝！衝！衝！

題目敘述

請寫一個程式輸出Hello World!

輸入格式

本題無需輸入值

輸出格式

A\~Z[a~z]、空格，以及常用英文符號。

資料範圍

A\~Z[a~z]、空格，以及驚嘆號“!”

測試範例

輸入範例 1

(無輸入值)

輸出範例 1

```
Hello World!
```

0_Hello World

(30 points)

Introduction

YTP Contest has started!

Let's verify everything first.

Is the internet setting correct?

Is the source code submission working well?

Do you use STDOUT output for program solutions?

Everything is ready! Go get 30 points now!! Go! Go! Go!

Statement

Please write a program to output Hello World!

Input Format

This problem requires no input.

Output Format

$A \setminus \sim Z[a \sim z]$, space, and common English punctuation.

Constraints

$A \setminus \sim Z[a \sim z]$, space, and exclamation mark "!".

Test Cases

Input 1

(no input)

Output 1

```
Hello World!
```

Illustrations

Input 1 has no input, simply output Hello World!

1_回聲壓縮(Echo Compression)

(10 分)

時間限制: 1.0 second

記憶體限制: 1024 MB

題目敘述

在資訊王國裡，每當兩個相同的字母連在一起說話，聲音就會產生「共鳴回聲」，聽起來像是：

aaabbcccaa → 啊啊啊啵啵嘻嘻嘻嘻啊啊

作為壓縮精靈，你的任務就是清除這些吵雜的回聲。

請你寫一個程式，幫忙把連續重複的字母簡化成一個，還原出清楚明瞭的訊息。

輸入格式

一行原始字串 s 。

輸出格式

輸出一行經過壓縮後的字串。

資料範圍

- s 僅由小寫英文字母組成
- s 的長度不超過 10^5
- s 不為空字串

子任務

- 子任務 1 (10 分): 無額外限制

測試範例

輸入範例 1

```
aaabbcccaa
```

輸出範例 1

```
abca
```

1_Echo Compression

(10 points)

Time Limit: 1.0 second

Memory Limit: 1024 MB

Statement

In the Kingdom of Signals, every time two or more identical letters come together, they resonate and echo endlessly. It sounds like:

```
"aaabbcccaa" -> "ahahah bibi cicici ahah".
```

As a echo compressor, your job is to remove this noisy repetition. For every group of adjacent identical characters, compress them into a single character, revealing the core message.

Input Format

A line of string s of initial message.

Output Format

Output the compressed version of the string.

Constraints

- s consists of only lowercase English letters
- the length of s is no more than 10^5
- s is not an empty string

Subtasks

- Subtask 1 (10 points): No additional constraints

Samples

Sample Input 1

```
aaabbcccaa
```

Sample Output 1

```
abca
```

2_除草劑(Herbicide)

(10 分)

時間限制: 1.0 second

記憶體限制: 1024 MB

題目敘述

田野裡有一片雜草叢生的地段，長度恰好是 n 格，每格上長著一棵高度介於 1 到 n 的野草，透過他仔細的觀察，他發現一定會有一個雜草長得特別矮，高度只有 1。某天，農夫 Willy 發明了一種神奇的除草劑，只要在一段連續格子區間 $[l, r]$ 上噴灑，並選擇強度 k ($1 \leq k \leq n$)，該區間內每棵野草的高度 A_i 就會瞬間變成：

$$A_i \leftarrow \left\lfloor \frac{A_i}{k} \right\rfloor \quad (1)$$

其中高斯符號 (floor function) 的定義為：

$\lfloor x \rfloor =$ 不大於 x 的最大整數

例如：

- $\lfloor 2.4 \rfloor = 2$
- $\lfloor -2.4 \rfloor = -3$
- $\lfloor 7 \rfloor = 7$
- $\lfloor \pi \rfloor = 3$

Willy 的目標，是將整塊地段上的所有雜草高度修整至 **完全相同**。請你幫助他，計算最少需要施放除草劑多少次，以及具體的操作方案。

輸入格式

第一行輸入一個正整數 n ，表示雜草的數量。

第二行輸入 n 個正整數 a_i ，表示第 i 格雜草的初始高度。

輸出格式

第一行輸出最少施放除草劑的次數 p ，必須滿足 $0 \leq p \leq N$ 。

接下來輸出 p 行，每一行包含 l, r, k ，代表你選擇了強度為 k 的除草劑，並且施放除草劑到 $[l, r]$ 這個區間。
 $1 \leq l, r, k \leq n$

資料範圍

- $1 \leq N \leq 10^5$
- $1 \leq a_i \leq N$

子任務

- 子任務 1 (10 分): 無額外限制

測試範例

輸入範例 1

```
2
1 2
```

輸出範例 1

```
1
2 2 2
```

輸入範例 2

```
4
1 4 2 1
```

輸出範例 2

```
2
2 2 4
3 3 2
```

範例說明

在第一筆測資中，可以用強度為 2 的除草劑讓第二個雜草變成高度 1。

在第二筆測資中，由於不可能用一次以下操作達成條件，因此分別用一次操作把 a_2, a_3 變成 1。

下面這也是一個正確的輸出：

```
2
2 3 2
2 2 2
```

2_Herbicide

(10 points)

Time Limit: 1.0 second

Memory Limit: 1024 MB

Statement

On a grassy field divided into n plots in a single line, each plot contains a weed whose height is an integer between 1 and n . Through his careful observation, he noticed that there is always one weed that is particularly short, with a height of only 1. Farmer Willy has invented a magical herbicide. When he sprays it over a contiguous segment of plots $[l, r]$ and chooses an power k ($1 \leq k \leq n$), the height of every weed in that segment instantly becomes

$$A_i \leftarrow \left\lfloor \frac{A_i}{k} \right\rfloor. \quad (2)$$

The definition of the Gauss symbol (floor function) is:

$\lfloor x \rfloor$ = the greatest integer less than or equal to x

For example:

- $\lfloor 2.4 \rfloor = 2$
- $\lfloor -2.4 \rfloor = -3$
- $\lfloor 7 \rfloor = 7$
- $\lfloor \pi \rfloor = 3$

Willy's goal is to make all weeds across the entire field have **exactly the same height**.

Your task is to determine the minimum number of herbicide applications required and provide a sequence of operations that achieves this goal.

Input Format

The first line contains a positive integer n , representing the number of weeds.

The second line contains n positive integers a_i , where a_i is the initial height of the weed in the i -th plot.

Output Format

The first line prints an integer p , the minimum number of herbicide applications, $0 \leq p \leq N$ must holds.

The next p lines each contain three integers l, r, k .

Each line describes one operation: spray herbicide of strength k over the segment $[l, r]$. $1 \leq l, r, k \leq n$

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq a_i \leq N$

Subtasks

- Subtask 1 (10 points): No additional constraints

Samples

Sample Input 1

```
2
1 2
```

Sample Output 1

```
1
2 2 2
```

Sample Input 2

```
4
1 4 2 1
```

Sample Output 2

```
2
2 2 4
3 3 2
```

Illustrations

In sample testcases:

Test case 1:

Using herbicide of strength 2, you can reduce the height of the second weed to 1.

Test case 2:

Because the goal cannot be achieved with a single operation, apply the herbicide two times to set a_2 to 1 and set a_3 to 1.

Below is also a correct output:

```
2
2 3 2
2 2 2
```

3_彩色路徑(Rainbow Path)

(6 分 / 9 分)

時間限制: 1.0 second

記憶體限制: 1024 MB

題目敘述

在一個小農村裡，農夫們喜歡栽種彩色的花朵，其中每個農夫會將他的農場裡都種滿同一種顏色的花朵。由於地區發展問題，因此並不是每個農場都是可以直接相通的，有時候需要先經過其他農場繞路，或是有時候甚至沒有道路可以到達。今天市長來訪查，一共會走過三個農場 u, v, w ，農村裡的人們希望讓市長可以走過**花朵顏色都不相同**，並且相互相鄰連成一條連續路徑的三個農場（也就是說， c_u, c_v, c_w 兩兩相異，且 (u, v) 跟 (v, w) 之間都有直接相連的道路），並把這個探訪路徑 $u \rightarrow v \rightarrow w$ 稱為彩色路徑，因此他們想要統計農村裡共有多少個彩色路徑，但是彩色路徑數量可能會非常多，因此他們請求你幫他們計算。

由於發展經費問題，因此農村裡不會蓋設無意義的道路，也就是不會有道路兩端連接一樣的農場，也不會有兩條不同道路連接一樣的起點與終點。

請注意 $a \rightarrow b \rightarrow c$ 和 $c \rightarrow b \rightarrow a$ 會視作兩條不同路徑。

輸入格式

第一行會輸入兩個正整數 N 和 M ，代表農場的數量還有道路的數量。

第二行會輸入 N 個正整數 c_i ，代表每個農場的花朵顏色。

接下來會輸入 M 行，每行有兩個數字 a_i, b_i ，代表一條雙向的道路連接的兩個農場。

輸出格式

輸出一個數字代表總共有幾行彩虹路徑。

保證路徑的數量不會超過 10^{18} 。

資料範圍

- $1 \leq N, M \leq 3 \times 10^5$
- $1 \leq c_i \leq N$
- $1 \leq a_i, b_i \leq N$
- 對於所有 i , $a_i \neq b_i$
- 對於所有 $i \neq j$, $\{a_i, b_i\} \neq \{a_j, b_j\}$

子任務

- 子任務 1 (6 分): $N \leq 2000, M \leq 2000$
- 子任務 2 (9 分): 無額外限制

測試範例

輸入範例 1

```
3 2
1 2 3
1 2
2 3
```

輸出範例 1

```
2
```

輸入範例 2

```
5 4
1 1 1 1 1
1 2
2 3
3 4
4 5
```

輸出範例 2

```
0
```

輸入範例 3

```
5 7
1 2 3 4 1
1 2
2 3
3 4
4 5
1 3
3 5
2 5
```

輸出範例 3

24

範例說明

範例測資一中:

1-2-3 以及 3-2-1 為兩條彩虹路徑，因此輸出 2。

範例測資二中:

由於全部的顏色都是 1，因此沒有彩虹路徑，輸出 0。

3_Rainbow Path

(6 points / 9 points)

Time Limit: 1.0 second

Memory Limit: 1024 MB

Statement

In a small rural village, farmers enjoy planting fields of colorful flowers. Each farmer fills their entire farm with flowers of a single color. Due to development issues, not every farm is directly connected --- sometimes one must travel through other farms, or there may be no path at all. Today, the mayor will visit exactly three farms in sequence. The villagers want the mayor to walk through three farms whose **flower colors are all different** (In other words, c_u, c_v , and c_w are pairwise distinct, and there are direct roads connecting u to v and v to w). We call the visiting path $u \rightarrow v \rightarrow w$ a colored path. They need you to count how many rainbow paths exist in the village.

Due to limited development funds, pointless roads aren't built in the countryside --- that is, there are no roads whose two ends connect to the same farm, nor are there two different roads connecting the same pair of farms.

Note that $a \rightarrow b \rightarrow c$ and $c \rightarrow b \rightarrow a$ are considered two distinct paths.

Input Format

The first line contains two positive integers N and M , representing the number of farms and the number of roads.

The second line contains N positive integers c_i , the flower color at the i -th farm.

Each of the next M lines contains two integers a_i and b_i , indicating an undirected road between farms a_i and b_i .

Output Format

Print a single integer: the total number of colorful paths of length two (three farms) in the graph.

It is guaranteed that this number will not exceed 10^{18} .

Constraints

- $1 \leq N, M \leq 3 \times 10^5$
- $1 \leq c_i \leq N$
- $1 \leq a_i, b_i \leq N$
- $a_i \neq b_i$ for all i
- $\{a_i, b_i\} \neq \{a_j, b_j\}$ when $i \neq j$

Subtasks

- Subtask 1 (6 points): $N \leq 2000, M \leq 2000$
- Subtask 2 (9 points): No additional constraints

Samples

Sample Input 1

```
3 2
1 2 3
1 2
2 3
```

Sample Output 1

```
2
```

Sample Input 2

```
5 4
1 1 1 1 1
1 2
2 3
3 4
4 5
```

Sample Output 2

```
0
```

Sample Input 3

```
5 7
1 2 3 4 1
1 2
2 3
3 4
4 5
1 3
3 5
2 5
```

Sample Output 3

```
24
```

Illustrations

In example testcase 1:

1-2-3 and 3-2-1 are two rainbow paths, so output 2.

In example testcase 2:

The colors of flower in farms are all the same, so output 0.

4_彩色路徑(Reflecmino)

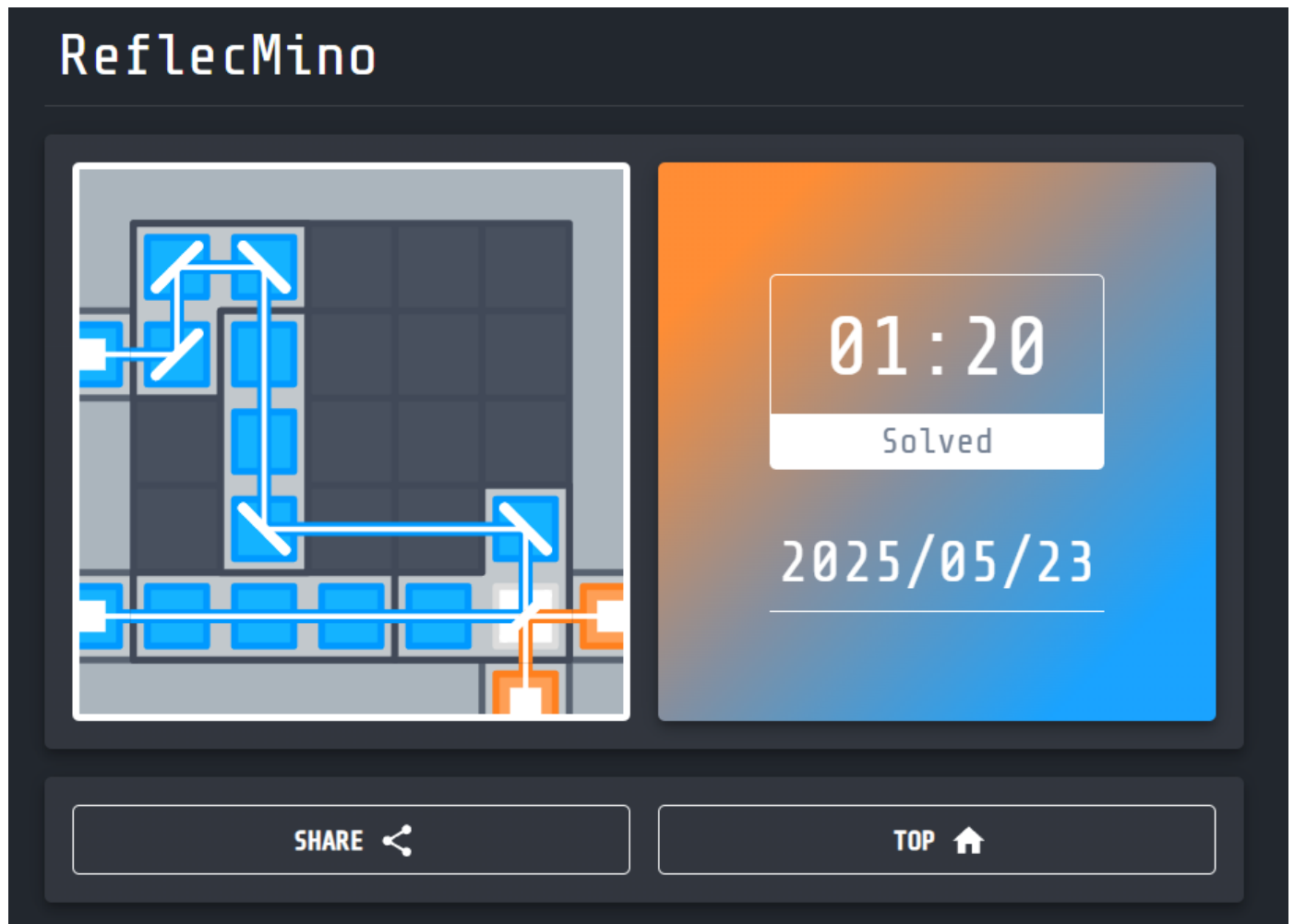
(15 分)

時間限制: 1.5 second

記憶體限制: 1024 MB

題目敘述

Mango 很喜歡玩一款叫做 ReflecMino 的遊戲，遊戲裡面會有兩條光線，並且要擺放包含平面鏡的積木，讓光線經過格子和平面鏡到達邊界的終點。以下是遊戲畫面：



遊戲的時候，會顯示藍色光線經過的格子以及橘色光線經過的格子

如果橘色和藍色的格子同時經過同一個格子 (像右下角) 則會顯示為白色

Mango 想要寫一個程式，讓他能夠模擬現在的地圖中平面鏡的所有擺放位置和角度，並且計算出光線的行走路線，為了簡化，他把積木的元素去除了，只留下平面鏡和格子。

現在在一個 $n \times m$ 的方格世界外圍，各有一個入射點：B（藍色光束）與 G（橘色光束）。格子中有三種物件：

-  與 ：平面鏡，兩者均為 45° 角鏡面，滿足「入射角 = 反射角」；
- ：空白，不改變光束方向；
- 邊界上恰有一個  和一個 。

光束從邊界入射後，每步沿當前方向前進一格：

1. 遇到空白 `.` 保持原方向；
2. 遇到鏡子 `\` 或 `/`，依鏡面方向反射；
3. 再次到達邊界即離開。

輸入格式

- 第一行為兩個整數 n, m 。
- 接下來 $n + 2$ 行，每行 $m + 2$ 個字元：
 - 邊界中恰有一個 `B` 和一個 `G`；
 - 其他位置為鏡面 `\`、鏡面 `/` 或空白 `.`。

輸出格式

在內層格子中：

- 僅被藍色光束經過的格子標記 `B`；
- 僅被橘色光束經過的格子標記 `G`；
- 同一格被兩束光都經過則標記 `W`（白光）；
- 未被任何光束經過的格子標記 `.`。

輸出整個內層格子，地圖大小為 $n \times m$

資料範圍

- $1 \leq n, m \leq 1000$
- 題目保證不會出現光線在鏡子形成無限迴圈反射。
- 題目保證不會有鏡子出現在邊界上或角落上
- 保證 `B`, `G` 會出現在邊界上，且不會在角落

子任務

- 子任務 1 (15 分): 無額外限制

測試範例

輸入範例 1

```

5 5
....B..
....\\.
G..\\./.
././\\.
.....
.\./...
.....

```

輸出範例 1

```

..BBB
GGWBB
GGGGG
G.G.G
GGG.G

```

輸入範例 2

```

3 3
.....
.....
B..\.
././.
.G...

```

輸出範例 2

```

...
WWW
WWW

```

4_Reflecmino

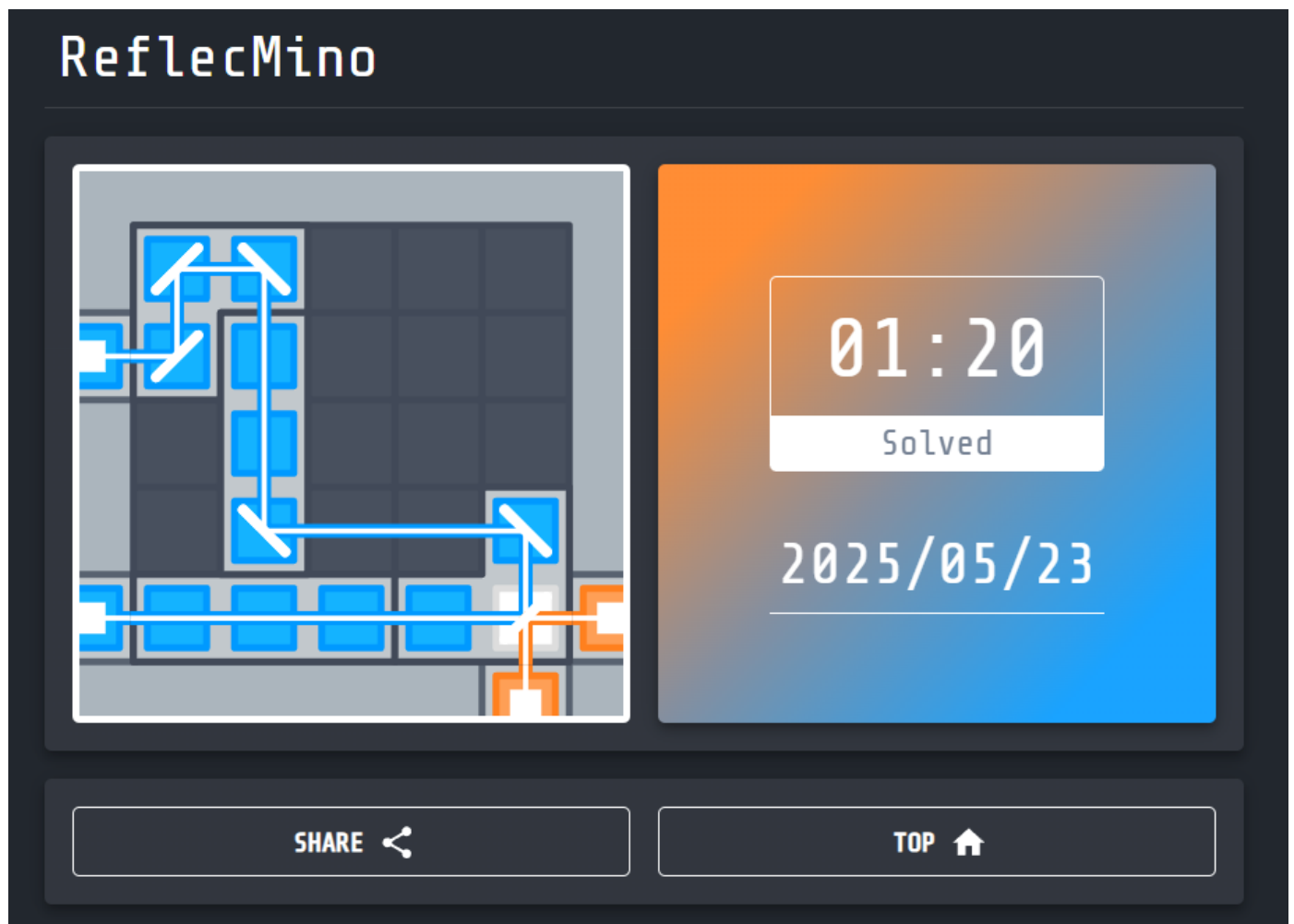
(15 points)

Time Limit: 1.5 second

Memory Limit: 1024 MB

Statement

Mango loves playing a game called ReflecMino. In this game, two beams of light travel through a grid filled with mirrors, and you must place the mirrors so that each beam passes through cells and reflects off mirrors to reach the boundary exit. Here is a screenshot of the game:



During the game, it highlights the cells traversed by the blue beam and those traversed by the orange beam.

If both the blue and orange beams pass through the same cell (as shown in the bottom right), that cell lights up white.

Mango wants to write a program that, given a map of mirror placements and orientations, simulates both light beams' paths. To simplify, he has removed all other game elements and only left mirrors and empty cells.

Now consider an $n \times m$ grid with an extra boundary layer. There are two entry points on that boundary: **B** (blue beam) and **G** (orange beam). Each cell contains one of:

- `\` or `/`: a flat mirror at 45°, reflecting light according to the law of reflection (“angle in = angle out”);
- `.`: an empty cell that does not change the beam’s direction;
- The boundary cells contain exactly one `B` and one `G`.

A beam starts from its entry point and moves one cell at a time in its current direction:

1. On an empty `.` cell, it continues straight;
2. On a mirror (`\` or `/`), it reflects accordingly;
3. When it reaches a boundary cell again, it exits.

Input Format

- The first line has two integers n, m .
- The next $n + 2$ lines each contain $m + 2$ characters:
 - The boundary contains exactly one `B` and one `G`.
 - Other positions are a mirror `\`, a mirror `/`, or an empty cell `.`.

Output Format

Within the inner $n \times m$ grid, mark each cell as follows:

- `B` if only the blue beam passes through;
- `G` if only the orange beam passes through;
- `W` (white) if both beams pass through the same cell;
- `.` if neither beam passes through.

Output the entire inner grid, which is of size $n \times m$.

Constraints

- $1 \leq n, m \leq 1000$
- It is guaranteed that no light beam will ever get trapped in an infinite reflection loop among the mirrors.
- The problem guarantees that no mirror will appear on the boundary or in the corners of the board.
- There is exactly one `B` and one `G` on the boundary, and neither of them is placed in a corner.

Subtasks

- Subtask 1 (15 points): No additional constraints

Samples

Sample Input 1

```

5 5
....B..
....\\.
G..\\./.
././\\.
.....
.\./...
.....

```

Sample Output 1

```

..BBB
GGWBB
GGGGG
G.G.G
GGG.G

```

Sample Input 2

```

3 3
.....
.....
B..\.
././.
.G...

```

Sample Output 2

```

...
WWW
WWW

```

5_魔法蛇神之儀式(The Ritual of the Serpent God)

(4 分 / 8 分 / 8 分)

時間限制: 3.0 second

記憶體限制: 2048 MB

題目敘述

在遠古的奧爾托利亞大陸，傳說有一座名為「蛇神之塔」的神殿。

祭司們為了向蛇神祈雨，會舉行一種奇特的「魔法蛇舞儀式」：

- 他們會用一串由 n 段「靈蛇節」串接而成的魔法蛇，從地面蜿蜒向空中舞動。
- 每一段靈蛇節是由純能量凝聚而成，長度為 L_i 。
- 每一段靈蛇節的相對轉角 θ_i 可以設定為 $0^\circ, 90^\circ, 180^\circ, 270^\circ$ 四種固定角度（因為這是蛇神咒文中允許的穩定魔法角度）。
- 靈蛇是從原點 $(0, 0)$ 出發，初始沿著 $+x$ 軸方向延展。
- 每一段靈蛇節會先相對上一段旋轉逆時針 θ_i ，然後沿著當前方向平移 L_i 長度。若為第一段靈蛇節，則直接從原點 $(0, 0)$ 以 θ_1 角度（ 0° 為 $+x$ 軸， 90° 為 $+y$ 軸， 180° 為 $-x$ 軸， 270° 為 $-y$ 軸）出發，並平移 L_1 長度。

在儀式進行過程中，祭司們可以不斷調整靈蛇節的轉角，或者查詢當前靈蛇在前 k 段之後的末端位置，藉此判斷儀式進展是否成功。

一開始 θ_i 全部設為 0° 。儀式過程中有 q 次操作，分為以下三種：

1. 觀測靈蛇

`1 k` — 查詢前 k 段靈蛇節合成之後的末端位置 (x, y) 。

2. 變更咒文（改變角度）

`2 x theta` — 將第 x 段靈蛇節的轉角設為 theta° ，其中 $\text{theta} \in \{0, 90, 180, 270\}$ 。

3. 變更咒文（改變長度）

`3 x len` - 將第 x 段靈蛇節的長度改為 len ，其中 len 為正整數。

換句話說，你需要計算從原點出發，按照前 k 段當前設定的旋轉和平移，最終位置會到哪裡。

輸入格式

輸入的第一行包含兩個正整數 n, q ，代表靈蛇節的數量和操作的次數。

第二行包含 n 個正整數 $L_1, L_2, \dots, L_n$ ，代表每段靈蛇節的長度。

接下來的 q 行，每行包含一個操作，格式如下：

- `1 k`：查詢前 k 段靈蛇節合成之後的末端位置。
- `2 x theta`：將第 x 段靈蛇節的轉角改為 theta° ，其中 $\text{theta} \in \{0, 90, 180, 270\}$ 且 $1 \leq x \leq n$ 。
- `3 x len`：將第 x 段靈蛇節的長度改為 len ，其中 $1 \leq \text{len} \leq 10^9$ 且 $1 \leq x \leq n$ 。

輸出格式

對於每個查詢操作 `1 k`，輸出一行，包含兩個整數 x 和 y ，代表前 k 段靈蛇節合成之後的末端位置 (x, y) 。

資料範圍

- $1 \leq n, q \leq 2 \times 10^5$ 。
- $1 \leq L_i \leq 10^9$ ($1 \leq i \leq n$)。
- 對於每個操作：
 - `1 k`，其中 $1 \leq k \leq n$ 。
 - `2 x theta`，其中 $\text{theta} \in \{0, 90, 180, 270\}$ 。
 - `3 x len`，其中 $1 \leq \text{len} \leq 10^9$ 。

子任務

- 子任務 1 (4 分): $1 \leq n, q \leq 2 \times 10^3$
- 子任務 2 (8 分): 對於每個指令 `2 x theta` 的 $\text{theta} \in \{0, 180\}$
- 子任務 3 (8 分): 無額外限制

測試範例

輸入範例 1

```
3 5
1 2 8
1 3
2 3 180
1 3
3 2 1
1 3
```

輸出範例 1

```
11 0
-5 0
-6 0
```

輸入範例 2

```
4 5
2 3 1 4
1 4
2 2 90
3 1 1
3 2 1
1 4
```

輸出範例 2

```
10 0
1 6
```

輸入範例 3

```
4 6
1 1 1 1
1 4
2 1 90
2 3 90
2 2 90
2 4 90
1 4
```

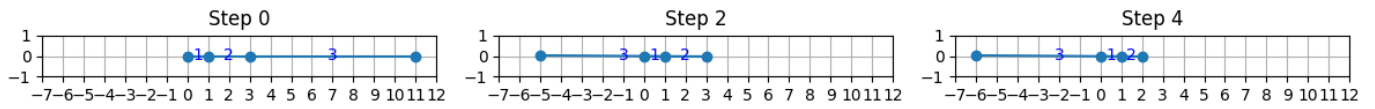
輸出範例 3

```
4 0
0 0
```

範例說明

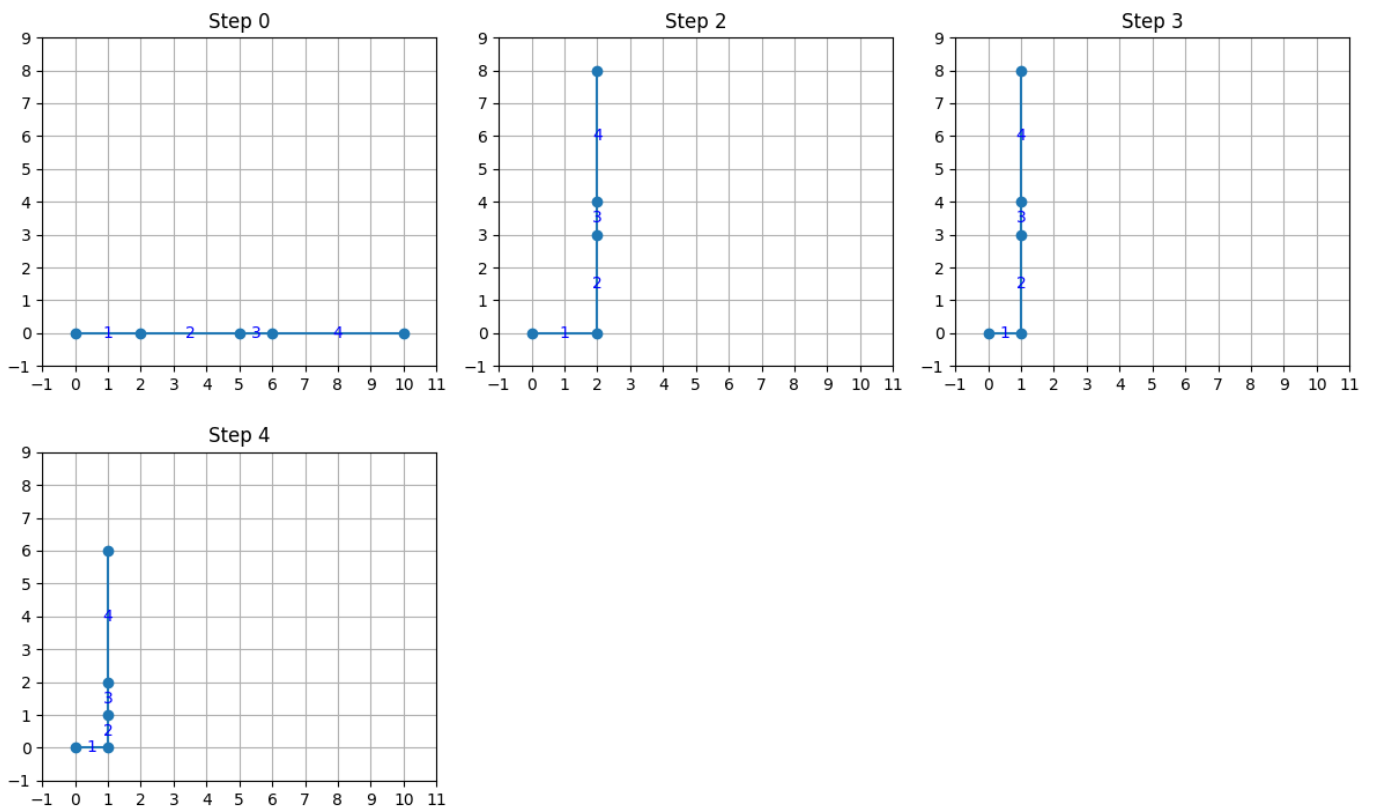
範例 1 的每一步操作後的結果如下圖所示：

Evolution of the Magic Snake



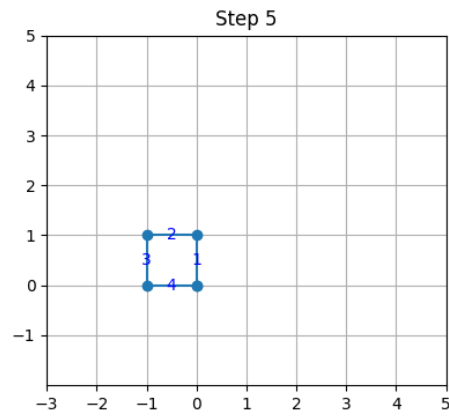
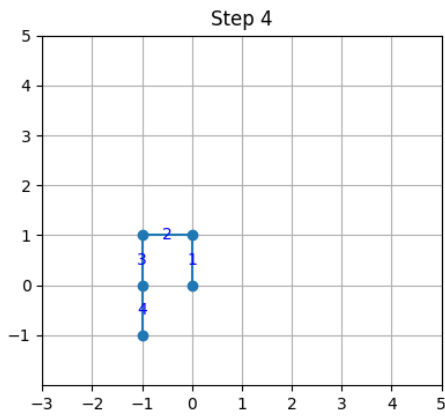
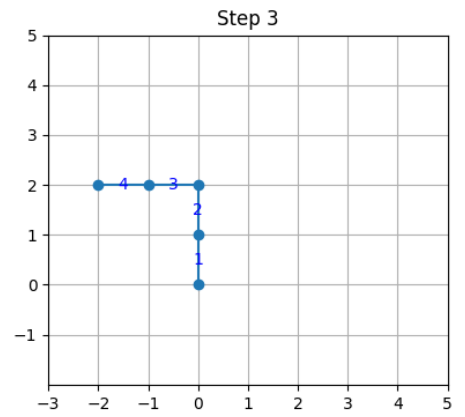
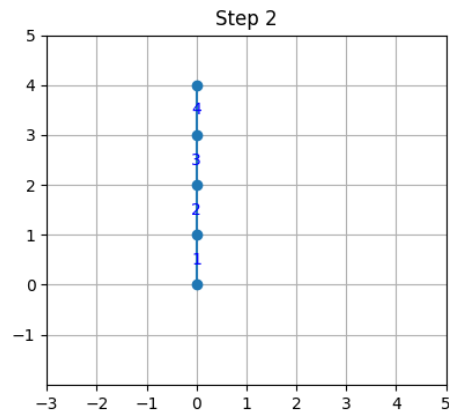
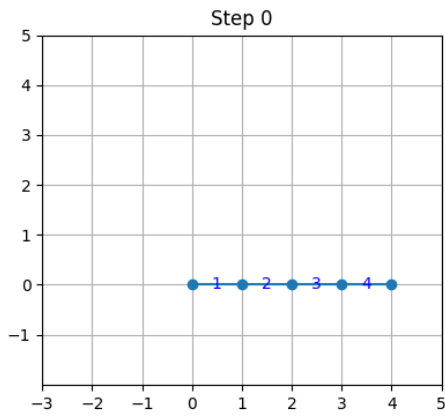
範例 2 的每一步操作後的結果如下圖所示：

Evolution of the Magic Snake



範例 3 的每一步操作後的結果如下圖所示：

Evolution of the Magic Snake



5_The Ritual of the Serpent God

(4 points / 8 points / 8 points)

Time Limit: 3.0 second

Memory Limit: 2048 MB

Statement

In the ancient land of Ortoloria, there is a legendary temple known as the Tower of the Serpent God.

The high priests, seeking rain from the serpent god, perform a mysterious ritual called the Dance of the Magic Serpent:

- They construct a long magical serpent composed of n serpent segments, which weaves through the air starting from the ground.
- Each serpent segment is made of pure magical energy and has a length L_i .
- The relative angle of each serpent segment θ_i can be set to one of four stable magic angles: 0° , 90° , 180° , or 270° — as prescribed by the sacred incantation.
- The serpent starts from the origin point $(0, 0)$ on the ground.
- For each subsequent segment, it first rotates counterclockwise by θ_i relative to the previous segment's direction, then moves forward by L_i in the new direction. For the first serpent segment, it extends from the origin at angle θ_1 (0° is along the $+x$ axis, 90° is along the $+y$ axis, 180° is along the $-x$ axis, and 270° is along the $-y$ axis), moving forward by L_1 .

During the ritual, the priests can dynamically adjust the angles of the serpent segments or observe the current position of the serpent's head after the first k segments to gauge the success of the ritual.

Initially, all angles θ_i are set to 0° . There will be q operations performed during the ritual, of the following three types:

1. Observe the Serpent
`1 k` — Query the end position (x, y) of the serpent's after the first k segments.
2. Change Incantation (Angle)
`2 x theta` — Change the angle of the x -th serpent segment to theta° , where $\text{theta} \in 0, 90, 180, 270$.
3. Change Incantation (Length)
`3 x len` — Change the length of the x -th serpent segment to len , where len is a positive integer.

In other words, you must compute the position starting from the origin, after applying the rotations and translations of the first k segments according to their current configurations.

Input Format

The first line contains two positive integers n and q , representing the number of serpent segments and the number of operations.

The second line contains n positive integers $L_1, L_2, \dots, L_n$, representing the initial lengths of the serpent segments.

The next q lines each describe an operation in one of the following formats:

- `1 k`: Query the end position of the serpent's after the first k segments.
- `2 x theta`: Set the angle of the x -th segment to theta° , where $\text{theta} \in \{0, 90, 180, 270\}$ and $1 \leq x \leq n$.
- `3 x len`: Change the length of the x -th segment to len , where $1 \leq \text{len} \leq 10^9$ and $1 \leq x \leq n$.

Output Format

For each `1 k` query, output one line containing two integers x and y , representing the end position (x, y) of the serpent after the first k segments.

Constraints

- $1 \leq n, q \leq 2 \times 10^5$.
- $1 \leq L_i \leq 10^9$ ($1 \leq i \leq n$).
- For each operation:
 - `1 k`, where $1 \leq k \leq n$.
 - `2 x theta`, where $\text{theta} \in \{0, 90, 180, 270\}$.
 - `3 x len`, where $1 \leq \text{len} \leq 10^9$.

Subtasks

- Subtask 1 (4 points): $1 \leq n, q \leq 2 \times 10^3$
- Subtask 2 (8 points): For each command `2 x theta`, $\text{theta} \in \{0, 180\}$
- Subtask 3 (8 points): No additional constraints

Samples

Sample Input 1

```
3 5
1 2 8
1 3
2 3 180
1 3
3 2 1
1 3
```

Sample Output 1

```
11 0
-5 0
-6 0
```

Sample Input 2

```
4 5
2 3 1 4
1 4
2 2 90
3 1 1
3 2 1
1 4
```

Sample Output 2

```
10 0
1 6
```

Sample Input 3

```
4 6
1 1 1 1
1 4
2 1 90
2 3 90
2 2 90
2 4 90
1 4
```

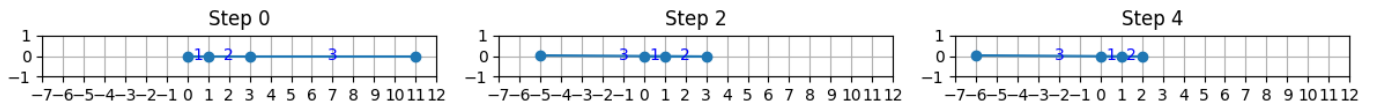
Sample Output 3

```
4 0
0 0
```

Illustrations

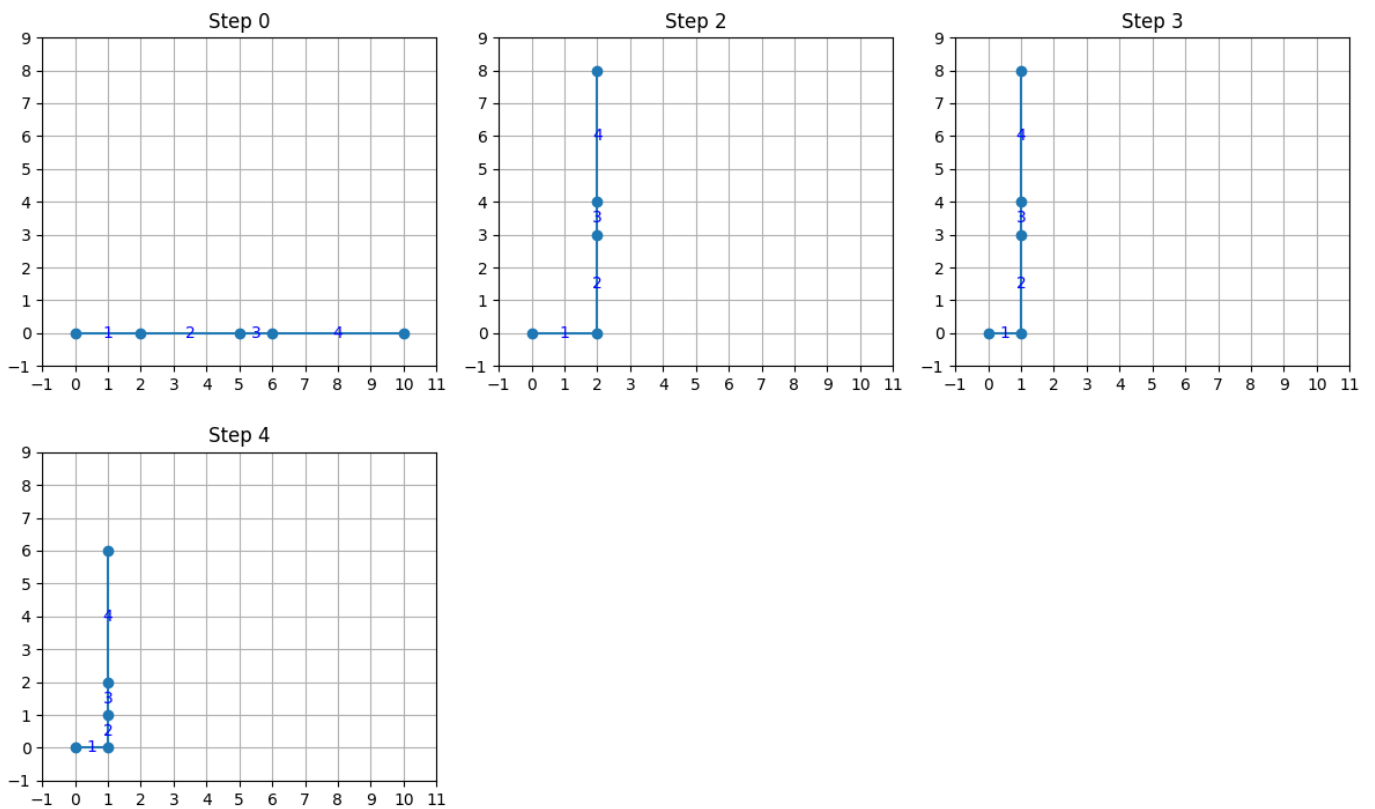
The results of each operation in Example 1 are illustrated in the following diagram:

Evolution of the Magic Snake



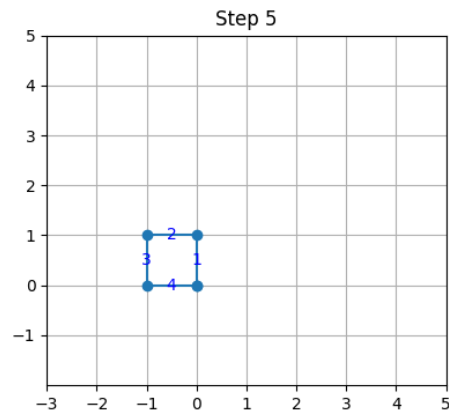
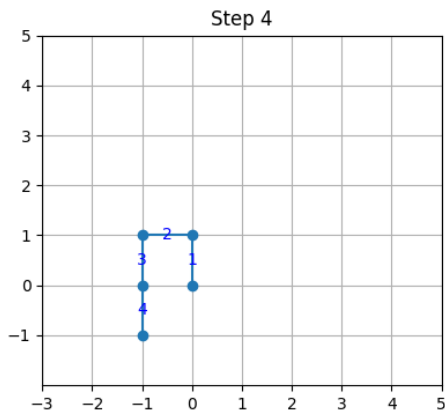
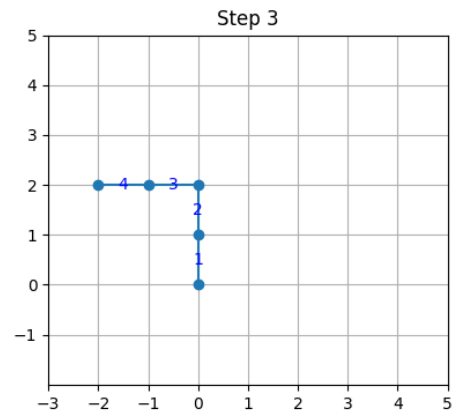
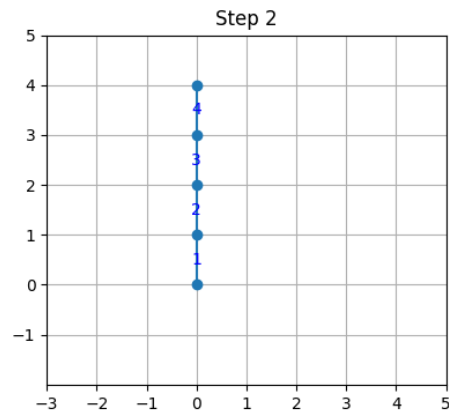
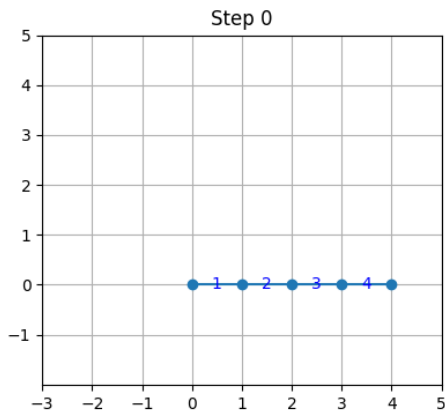
The results of each operation in Example 2 are illustrated in the following diagram:

Evolution of the Magic Snake



The results of each operation in Example 3 are illustrated in the following diagram:

Evolution of the Magic Snake



6_四皇后問題(Four Queens Puzzle)

(10 分 / 10 分)

時間限制: 4.0 second

記憶體限制: 1024 MB

題目敘述

皇后是西洋棋中非常強大的棋子，它可以沿著八種方向在方格上移動到任意格子直到被棋子或邊界擋住，如果受到阻擋的對象是一個對手的棋子，那皇后可以移動到那個格子並吃掉他。

正式來說，一枚在格子 (x, y) 的皇后可以沿著

$(1, 0), (1, 1), (0, 1), (-1, 1), (-1, 0), (-1, -1), (0, -1), (1, -1)$ 的其中一個方向 (a, b) 移動一個整數倍數 d 如果：

1. 格子 $(x + ad, y + bd)$ 是空的或被對面的棋子佔領。
2. 如果 $d > 1$ ，那對於所有 $1 \leq c < d$ ，格子 $(x + ac, y + bc)$ 是空的。

在這個情況中，我們說格子 $(x + ad, y + bd)$ 在它的攻擊範圍內。

你的朋友又輸掉一場西洋棋錦標賽的資格賽，已經是連續第五十次了，

「這沒道理！」他的皇后動都沒動就被吃掉了，現在他破防到相信皇后很強只是大家一直道聽塗說，實際上根本就沒有。

你想要證明他的陰謀論是錯的，但他出了一道謎題給你。

在 $n \times m$ 的棋盤上，有數個阻擋格子的障礙物，你需要放置剛好四枚皇后，使得每一對皇后（總共有六對）都在彼此的攻擊範圍內，而且不會被其他皇后或障礙物阻擋。

為什麼是四枚？只是因為是數量最多的可能，不過這對身為西洋棋大師的你不是難題，找出有多少種放置方法可以滿足條件來否決你朋友的陰謀論。

輸入格式

輸入的第一行包含兩個整數 n, m 代表棋盤的高度與寬度。

接下來有 n 行，每行有一個長度為 m 的字串。

所有字串都由 e 與 . 組成，第 i 列第 j 個字元代表格子 (i, j) 的狀態： e 代表被障礙阻擋而 . 代表空的格子。

輸出格式

輸出有多少種放置方法可以滿足題目中的條件。

資料範圍

- $1 \leq n, m \leq 700$

子任務

- 子任務 1 (10 分): $n, m \leq 100$
- 子任務 2 (10 分): 無額外限制

測試範例

輸入範例 1

```
3 4
. @ @ @
....
...@
```

輸出範例 1

```
2
```

輸入範例 2

```
5 6
@.....
.....
....@.
.....
@.....
```

輸出範例 2

```
27
```

輸入範例 3

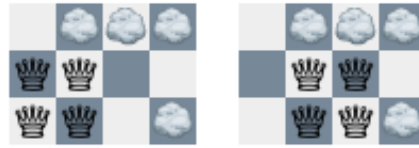
```
6 6
..@...
.....
.@....
.....
.....@
.....
```

輸出範例 3

```
35
```

範例說明

在第一筆範例中，唯二的放置可能是：



6_Four Queens Puzzle

(10 points / 10 points)

Time Limit: 4.0 second

Memory Limit: 1024 MB

Statement

Queens are very powerful pieces in chess.

A queen can move along one of the eight directions on a grid until that direction is blocked.

If a piece of the opposing team blocks it, then the queen can capture it.

Formally speaking, a queen at cell (x, y) can move along one of the eight directions

$(1, 0), (1, 1), (0, 1), (-1, 1), (-1, 0), (-1, -1), (0, -1), (1, -1)$ for any positive integer multiple d if, let (a, b) be the direction chosen:

1. The cell $(x + ad, y + bd)$ is on the board, either not blocked or occupied by another piece.
2. If $d > 1$, for every integer $1 \leq c < d$, the cell $(x + ac, y + bc)$ is empty.

In this case, we say the queen *controls* the cell $(x + ad, y + bd)$.

Your friend just lost another chess tournament... qualifier. Fiftieth in a row.

It is not fair!

His queen had never moved, ever, and got captured.

Now he is so tilted that he is starting to believe an absurd conspiracy theory: the queen is useful only because people say so. It is not.

You want to prove him wrong, but then he challenged you with a puzzle.

Place **four queens** on an $n \times m$ board, with several obstacles scattered around, blocking some of the cells.

Every pair of queens (there are six pairs in total) must mutually control each other's squares, without being blocked by obstacles or other queens.

Every pair of queens (there are six pairs in total) must be able to attack each other, as if they were enemies, with no obstacles or other queens in between.

Why four, you may ask? Because that is simply the maximum possible.

Nevertheless, for a chess master like you, this should be easy.

Count the number of such configurations to prove your friend wrong.

Input Format

The first line of the input contains two integers n and m representing the height and width of the board.

Next, n lines follow, each containing a string of length m .

Each string consists of `@` and `.`.

In the i -th row, the j -th character denotes the status of the cell (i, j) on the board: `@` for an obstacle and `.` for an empty cell.

Output Format

Output the total number of configurations satisfying the conditions in the description.

Constraints

- $1 \leq n, m \leq 700$

Subtasks

- Subtask 1 (10 points): $n, m \leq 100$
- Subtask 2 (10 points): No additional constraints

Samples

Sample Input 1

```
3 4
. @ @ @
....
...@
```

Sample Output 1

```
2
```

Sample Input 2

```
5 6
@.....
.....
....@.
.....
@.....
```

Sample Output 2

```
27
```

Sample Input 3

```

6 6
..@...
.....
.@....
.....
.....@
.....

```

Sample Output 3

```

35

```

Illustrations

In the first sample, the two possible configurations are:

